Pre-Study

Axel Lindqvist, axlindq@kth

August 30, 2026

1 Classic Optimization Methods

Optimization methods are methods used to systematically optimize an objective function $f(x)$ defined over some domain $x \in \mathcal{R}^n$ and find the minimizer $x^* = arg\ min_x f(x)$. Classic optimization methods minimize this function by iteratively applying hand-crafted update rules, where based on the expert's experience and knowledge about the problem, the rules are written. These are often written from a theoretical perspective and thus for many of them have theoretical guarantees on their performance, like Gradient Descent, Newton's Method and Simplex Method.
However, because classical methods are built purely on theoretical foundations and only take into account the restrictions and mathematical properties of the problem, they often suffer from high computational costs, being stuck in local minima, and the effectiveness of the optimization being heavily influenced by the starting parameters.
As a running example, consider the LASSO problem, a linear regression problem. The objective function and minimizer becomes as shown in Eq. 1, where x^* is a n-dimensional vector we are trying to recover, which is assumed to be sparse, b being a m-dimensional vector of observed measurements from x^* and the measurement matrix $A \in R^{m \times n}$. With the second term, the coefficient λ and L_1-regularization, promoting sparseness and the first term promoting our solution to be as close to our known measurements as possible.

$$x^* = arg\ min_{x \in R^n} \frac{1}{2}||Ax - b||_2^2 + \lambda||x||_1 \tag{1}$$

A classic optimization method to solve this problem is the iterative shrinking threshold algorithm (ISTA), which is an application of the proximal gradient method to the problem. Where the optimization algorithm can be written as Eq. 3 [1]. $S_\eta(z)$ is a soft-thresholding operator, L is the maximum eigenvalue of A^TA, t is the number of iterations and η the gradient descent stepsize. Which then due to its theoretical roots can be shown to have a worst-case convergence rate of $O(\frac{1}{t})$.

$$S_\eta(z) = sign(z)max(|z| - \eta, 0) \tag{2}$$

$$x_{t+1} = S_{\frac{\lambda}{L}}(x_t - \frac{1}{L}A^T(Ax_t - b)) \tag{3}$$

2 The need for Learning to Optimize (L2O)

Learning to Optimize (L2O) is a different kind of optimization method, where instead of determining the optimization rules before beginning training, a model is trained whose objective is to learn the optimization rule itself, we call this model the optimizer. This model is trained on a distribution of optimization problems, with the optimization problems being called the optimizees and the distribution of optimizees being called the task distribution.

$$x_{t+1} = x_t - \alpha \nabla f(x_t) \tag{4}$$

A classical optimizer updates the optimizees x based on a hand-crafted rule, like gradient descent in Eq. 4. Where α is a fixed step size. An L2O optimizer replaces this update rule with Eq. 5 with a learned mapping [2]

$$x_{t+1} = x_t + m(z_t;\, \phi) \tag{5}$$

Where $m(z_t;\, \phi)$ is a neural network parametrized by ϕ and z_t which is its input for that iteration, which often includes x_t, $\nabla f(x_t)$ and information from previous iterations.
By letting this optimizer train on the task distribution, it shapes the optimization method according to the task distribution and the problem formula. Instead of just being adapted to the problem formula like classic optimization methods. This gives the optimizer the ability to discover "short cuts" when the task distribution is concentrated, thus being faster than classic optimization methods, and to provide higher-quality solutions [2].

3 Model-free L2O

In model-free L2O, the optimization method is just the optimizer without assuming the update rule takes the form of any particular formula, except that the updates are iterative [2].
This optimizer then, unlike classic optimization methods, requires an offline training phase, where the optimizer is trained on optimizees sampled from the task distribution $\mathcal{T}$. This offline training can be defined as a loss function to be minimized, Eq. 6:

$$\mathcal{L}(\phi) = E_{(\theta_0,\, f)\in\mathcal{T}} \left[\sum_{t=1}^{T} w_t f(\theta_t) \right], \quad \theta_{t+1} = \theta_t + m(z_t; \phi) \tag{6}$$

Where w_t are weighting coefficients and $T \in N$ the time span. A popular type of optimizer are recurrent neural networks (RNN), especially Long Short-Term Memory (LSTM) for their ability to handle long term dependencies [3]. For RNN optimizers, the update rule can be written as Eq. 7. Where h_t is the hidden state [3].

$$\begin{pmatrix} g_t \\ h_{t+1} \end{pmatrix} = m(\nabla f(\theta_t),\, h_t,\, \phi), \qquad \theta_{t+1} = \theta_t + g_t \tag{7}$$

When the optimizee increases in number of parameters, the hidden state h and number of parameters θ needed increases drastically, leading to poor

scalability. To handle this coordinatewise optimizing with shared-weights for each instance of the optimizer is often implemented [3, 2]. With each coordinate having its own hidden state and thus having better scaling.

Despite this, model-free L2O still faces the issue of scalability as the training still requires back propagation through time (BPTT), so activations must be stored for every coordinate times every unrolled step. Thus scaling with dimensionality and unroll length. [2].

They also lack convergence guarantees, as the optimizer learns everything from scratch, as well as requiring a large number of training samples [2].

4 Model-based L2O

Model-based L2O, are an approach where the optimizer also takes inspiration from classic optimization methods. In hopes to combat some of the problem's model-free L2O encounters. Such as the lack of convergence guarantees and large number of training samples. Most model-based L2O methods has one of two main approaches, plug and play (PnP) or algorithm unrolling.

In plug and play, instead of training the neural network on a distribution of optimizees during the offline training. It is instead trained offline on some other task, this already trained neural network then replaces a part of a classic optimization method (plugged into), which then immediately can be used. With the part of the optimization method that the neural network replaces being one of the proximal operators. For context, plug and play is not only a subcategory of L2O, but was originally about plugging in a hand-crafted denoiser, not a neural network, to replace one of the proximal operators, as done in [4].
The second approach, algorithm unrolling, the parameters in the iterative optimization algorithm the model is based on are made into a neural network where each iteration of the algorithm is a hidden layer [5]. With the depth of the network being the number of iterations the algorithm is truncated to and this neural network then being trained end-to-end.

An example of this is LISTA, which is algorithm unrolling applied to ISTA, where the fixed matrices are replaced with learnable ones as in Eq. 8

$$x_{t+1} = S_{\theta_t}(W_e\, b + W_s\, x_t) \tag{8}$$

Where W_e and W_s are the learnable weights, but taking $W_e \equiv \frac{A^T}{L}, W_s \equiv I - \frac{A^T A}{L}$ and $\theta \equiv \frac{\lambda}{L}$ gives us ISTA again in Eq. 3. But making these learnable gives it the opportunity to converge far faster. [2, 6].

More generally, model-based L2O needs fewer parameters than model-free L2O due to incorporating classic optimization methods, having a better more reliable performance on suitable problems for the classic optimization method they are based on, better interpretability and convergence guarantees [2]. Though they are less flexible than model-free L2O and requiring a more case-by-case design.

5 Open Challenges in L2O

Even though L2O has many promising aspects, it also has numerous challenges left, like all research areas. With some of these challenges being the ones in the subsections below.

5.1 The issue of scalability of model-free L2O methods

As mentioned in section 3, despite the coordinatewise shared-weight trick [3] which mitigates the problem of the hidden state increasing drastically with the size of the optimizee, the issue of scalability still remains due to the use of BPTT whose memory cost scales with both dimensionality and unrolling length. There is also the problem of training instability when having a too long unrolling length T. This is due to gradient explosion and vanishing through the long unrolled computational graph. However, having too short an unrolling length introduces truncation bias instead, which causes learned optimizers to exhibit instability and poor-quality solutions when applied to training optimizees [2].

For cases where the optimizee is a neural network, many solutions to this problem have been proposed. One approach, published in 2022, has been to constrain the network updates to a tiny subspace and then explore learning rules on top of it, which greatly reduces the number of trainable parameters and removes the scalability roadblock. The constraining of network updates has to be constructed per model family [7]. During 2021-2022 there has also been exploration of using ES-estimators instead of BPTT when meta-training optimizers for neural networks to bypass this problem [8, 9]. These were, however, found to have their own scaling problem with computation cost [10]. In 2025, a new paper was published using PES-estimators, a type of ES-estimator, along with task augmentation, learned scheduler, learned update rule and decoupled training to outperform the meta-generalization of prior large-scale learned optimizers such as VeLO [10] with far less computing cost [11]. All of these advances are validated exclusively on neural network training, so whether they transfer to other problem classes remains untested.

5.2 Progress on the theoretical foundation of L2O

The paper [2] states at the time the current theoretical efforts could be narrowed down to four essential questions. Capacity, trainability, generalization and interpretability.

Capacity: Do there exist model parameters for L2O models that provably outperforms traditional optimization algorithms for in distribution problems? And is there any safeguard mechanism to ensure that they perform at least as well in OoD tasks too? The most common safeguard mechanism at the publication of [2] was a fallback scheme that, at each iteration of the deployed L2O optimizer, checks whether the L2O step decreases some energy and ,if not, the step is replaced by a classical update instead. Then, due to the monotone energy decrease, convergence is guaranteed.

Since then, some papers have managed to instead make this guarantee a part for model-based L2O. Like making every point in the parameter space

corresponding to a convergant algorithm and thus making subpar update steps impossible [12], or by integrating the structural guarantees from a classic optimization method like done in [13].

Trainability: Given the existence of those parameters in the capacity questions, what training method should be used to obtain them and do they have convergence guarantees to the ideal parameters?
For this question, very limited theoretical work had been done due to the non-convexity of L2O training objectives, with the exception of some work done that proved that on single scalar step size for Gradient Descent on quadratic objectives, a log-transformed meta-objective makes meta-gradient descent provably converge [14].

Since then, necessary conditions that a convergent update rule must satisfy on convex problems have been derived, though satisfying them by itself neither guarantees that the algorithm's iterates converge, nor that training the model converges to good parameters [15]. Which then for quadratic problems was further developed on by proving the training convergence of NN-parametrized L2O models that learn Gradient Descent hyperparameters [16].

Generalization: Does the trained L2O model generalize to unseen optimizees from the training distribution (interpolation), and can it extrapolate beyond it?
As surveyed in [2], a relationship between the generalization gap (the difference between performance on training optimizees and expected performance on unseen ones from $\mathcal{T}$) and the number of training instances was used to estimate how many samples should be used. The gap's upper bound has also been derived via Rademacher complexity for quadratic optimization, signal reconstruction and sparse linear representation problems. Stability has also been found to be highly related to generalization, with some works analyzing the stability of their methods via, for example, Lipschitz-constant estimates.

While the progress above was about the generalization of the optimizer on optimizees, the generalization of the optimizee remained unstudied. Since then, by establishing an implicit connection between the local entropy and the Hessian and incorporating flatness-aware regularizers into the meta-training of the optimizer, a theoretical guarantee for the optimizee's generalization capabilities has been proven [17].

Interpretability: How can we explain what the L2O models have learned?
As surveyed in [2], some work has been made on this front for model-based L2O. For algorithm unrolling, unrolled iterative hard-treshhold has been shown to better adopt dictionary and has milder requirements of the restricted Isometry property. As well as the success of LISTA on the tradeoff between convergance speed and reconstruction accuracy as well as the weights in LISTA being shown to have low coherence with the dictionary and an analytic approach to calculate the weight being proposed. With the work on the area for model-free L2O being sparser though still progressing, with [18] advancing it by establishing a holistic symbolic representation and analysis framework and tool and L2O model called Scale-L2O. And with [19] discovering that model-free L2O has learned interpretable behavior such as momentum, gradient clipping, and learning rate adaptation, as well as show the dynamics and mechanism used for this by the

models.

Since 2022, some further progress has been made, with [20] proposing a new framework which enables the plug in trained neural network in PnP to become more interpretable while achieving state-of-the-art results.

5.3 Broader applications of model-based L2O

As surveyed by [2], empirical success of model-based L2O had been limited to a few instances of inverse problems and sparse optimization. Thus the need for exploring broader applications was suggested. Since the innovation of model-based L2O they have been applied to many different areas such as Image restoration and reconstruction, medical and biological imaging and wireless communication [2].

Algorithm Unrolling has also been used to solve large-scale linear programming problems at scale with promising improvements compared its at scale struggling benchmarks [21], as well as expanding into nonlinear trajectory optimization and predictive safety filter problems, outperforming state-of-the-art learned QP solvers [22].

6 Existing benchmarks for L2O

Before 2022, there was no existing benchmarking L2O platform, but this changed when [2] was published, implementing numerous model-based and model-free optimizers across convex sparse recovery, LASSO, Rastrigin and NN training. Since then, however, not much further progress or other alternative benchmarking platforms have been published. The code for L2O models that have been published instead just being tailor made for the specific experiments and models those papers have conducted.

7 Planned experiments

7.1 Experiment 1: Model-free L2O Scalability and Truncation

The first experiment planned is to further investigate model-free scalability issues and see the relationship between the truncation, T, and dimensionality of the optimizee. Investigating how the unroll length trades optimizer quality against memory/compute costs and training stability and how, if it does, does that tradeoff shift as optimizee dimensionality grows? For the experiment three model-free methods will be used, L2O-DM [3], L2O-RNNProp [23] and L2O-Scale [24] will be trained and evaluated on the LASSO problem, on differing dimensionality of the LASSO problem, along with different levels of unrolling steps T. With the configurations stated in Table. 1 This will then be measured against several metrics, maximum allocated and reserved memory, average time per iteration, total time, meta-loss $L(\phi)$, meta-gradient norm $||\frac{L}{d\phi}||$ to monitor the vanishing-gradient problem occurring in long unrolls. As L2O-Scale clips the meta-gradient, which the other two methods do not, both before and after versions will be measured. As to see when this clipping kicks in and how large of an impact it has. The modified relative loss Eq. (10) will also be

recorded to keep track of the quality of the optimizee. With the metric being normalized as the loss scales with dimensionality so that is important to be able to compare across problem dimensions. The units of the metrics being stated in Table. 2. Due to the number of total configurations being large, and the amount of computing cost/time being uncertain. A single complete run across all configurations will be run, then additional runs to each configuration will be iteratively added. Thus allowing for early detection of potential computational bottlenecks.

Table 1: Experiment 1 Configuration: Model-free L2O Scalability Study

Parameter	**Values**
L2O Methods	L2O-DM, L2O-RNNProp, L2O-Scale
Problem Type	LASSO
Problem Dimensions (m, n)	$(25, 50)$, $(50, 100)$, $(250, 500)$, $(500, 1000)$, $(5000, 10000)$
Regularization λ	0.005
Unrolling Steps T	$2, 4, 8, 16, 20, 30, 50, 75, 100$
Number of Runs	10 per configuration
Total Configurations	135

Table 2: Experiment 1 Metrics: A table showing the different metrics along with their unit of measurement.

Metric Category	**Metric**	**Unit**
Memory	Peak Allocated Memory	MB
	Peak Reserved Memory	MB
Optimization	Meta-loss $\mathcal{L}(\phi)$	scalar
	Meta-gradient Norm $\left\|\frac{\partial \mathcal{L}}{\partial \phi}\right\|$	scalar
	Modified relative loss (Eq. 10)	relative
Computational	Average Time per Iteration	ms
	Total time	s
Statistics per Metric	Mean, Std Dev	across 10 runs

7.2 Experiment 2: Effect of training-distribution width on ID quality and OOD robustness

The second experiment will investigate the in-distribution quality vs OOD robustness tradeoff, as when having a concentrated training distribution, L2O methods discover "short cuts" specific to that distribution, while typically failing for OoD problems [2]. Thus with this experiment the following question is trying to answer the following question: How does the concentration of the training task distribution trade ID quality against OOD robustness, and do model-free and model-based L2O families differ in this trade?

To try to answer this question for convex problems, the experiment will be done on the LASSO problem with varying levels of sparsity in the training data,

which will impact its concentration.

In this experiment both model-free and model-based L2O will be used. The model-free one's being L2O-dM and L2O-RNNProp and model-based ones being LISTA and ALISTA, all of which are already implemented in the benchmarking platform [2].

The training and dataset will be the same across methods, though model-free and model-based will be trained to its own standard, fixed budget for each configuration. Which to start with will be inspired by what they give in [2]. The configuration values can be seen in Table. 3.

For the metrics of this experiment, each method is measured by the following 3 metrics. First the raw suboptimality gap between the objective value at the final output and the optimal objective value f_q^* Eq. (9).

$$g_q(x) = f_q(x) - f_q^*, \tag{9}$$

Second by the modified relative loss of [2], where the optimal soltuions is generated by 2000 iterations of FISTA and the expectations are taken over the testing set for that sparsity, eq (10).

$$R_{f,\mathcal{Q}}(x) = \frac{\mathbb{E}_{q\sim\mathcal{Q}}\left[f_q(x) - f_q^*\right]}{\mathbb{E}_{q\sim\mathcal{Q}}\left[f_q^*\right]}. \tag{10}$$

The third being measuring the NMSE against the ground-truth signal x^* for signal recovery, Eq (11)

$$\mathrm{NMSE}(x) = 10\log_{10}\frac{\|x - x^*\|_2^2}{\|x^*\|_2^2}. \tag{11}$$

As a lot of different sparsity values also are tested, and to notice in case the sparsity is too high to be recoverable at all and thus that being the reason for deteriorating results instead of something with the methods, LASSO-optimal recovery error will also be measured. Where NMSE (Eq. (11)) but with the the minimizer x^* of the LASSO objective instead of the output of the method. So when the value starts increasing, it can be concluded that a perfect recovery becomes impossible.

7.3 Reproducibility and Variance

For both experiments, to ensure reproducibility, specific numbers of the seeds for the random number generators will be used. Along with multiple runs for each model and parameter combination to combat the uncertainty of results that comes from the high-variance of results in between different runs. With 10 runs per trial being my starting point. To be changed depending on computing cost needs and capabilities. With for each of the metrics mean and standard deviation or interquartile range, depending on which fits the variation more.

7.4 Planned Platform Extensions

Implementing a more general unified pipeline so it is easier in the future to extend the platform with new evaluation models, training, evaluation functions, and which ensure reproducibility and fairness. As the current codebase does not

Table 3: Experiment 2 Configuration: Parameters and values

Parameter	**Values**
L2O Methods	L2O-DM, L2O-RNNProp, LISTA, ALISTA
Problem Type	LASSO
Problem Dimensions (m, n)	$(210, 300)$
Regularization λ	0.005
Narrow-sparsity Training Set	$p = 0.1$
Medium-sparsity Training Set	$p \sim [0.075, 0.2]$ (per instance)
Wide-sparsity Training Set	$p \sim [0.05, 0.3]$ (per instance)
Training-set Size	$32,000$
Test-set Size (per sparsity dataset)	$1,280$
Test Sparsity	$[0.02, 0.05, 0.10, 0.15, 0.20, 0.30, 0.35, 0.40]$
Number of Runs	10 seeds per configuration
Total Configurations	12

Table 4: Experiment 2 Metrics: A table showing metrics along with their unit of measurement

Metric Category	**Metric**	**Unit**
Optimization	Suboptimality gap (Eq. 9)	scalar
	Modified relative loss (Eq. 10)	relative
	NMSE vs x_{true} (Eq. 11)	dB
LASSO-optimal recovery error	NMSE with $x = x^*$	dB
Statistics	Mean, Std Dev	across 10 runs

have a unified pipeline but instead is a agglomeration of separate frameworks with their own pipelines. Along with making the platform code more atomized and, reusable and easier for new people to modify. Along with adding functions to calculate the different metrics used in experiments 1 and 2 and data generator for them.

References

[1] Govind M. Chari. Solving the Lasso problem. `https://govindchari.com/blog/2023/solving-lasso/`, 2023. Accessed: 2026-06-08.

[2] Tianlong Chen, Xiaohan Chen, Wuyang Chen, Howard Heaton, Jialin Liu, Zhangyang Wang, and Wotao Yin. Learning to optimize: A primer and a benchmark, 2021.

[3] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent, 2016.

[4] Singanallur V. Venkatakrishnan, Charles A. Bouman, and Brendt Wohlberg. Plug-and-play priors for model based reconstruction. In *2013*

IEEE Global Conference on Signal and Information Processing, pages 945–948, 2013.

[5] Vishal Monga, Yuelong Li, and Yonina C. Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing, 2020.

[6] Karol Gregor and Yann LeCun. Learning fast approximations of sparse coding. In *Proceedings of the 27th International Conference on International Conference on Machine Learning*, ICML'10, page 399–406, Madison, WI, USA, 2010. Omnipress.

[7] Xuxi Chen, Tianlong Chen, Yu Cheng, Weizhu Chen, Ahmed Awadallah, and Zhangyang Wang. Scalable learning to optimize: A learned optimizer can train big models. In Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner, editors, *Computer Vision – ECCV 2022*, pages 389–405, Cham, 2022. Springer Nature Switzerland.

[8] Luke Metz, C. Daniel Freeman, James Harrison, Niru Maheswaranathan, and Jascha Sohl-dickstein. Practical tradeoffs between memory, compute, and performance in learned optimizers. In Sarath Chandar, Razvan Pascanu, and Doina Precup, editors, *Proceedings of The 1st Conference on Lifelong Learning Agents*, volume 199 of *Proceedings of Machine Learning Research*, pages 142–164. PMLR, 22–24 Aug 2022.

[9] Paul Vicol, Luke Metz, and Jascha Sohl-Dickstein. Unbiased gradient estimation in unrolled computation graphs with persistent evolution strategies, 2021.

[10] Luke Metz, James Harrison, C. Daniel Freeman, Amil Merchant, Lucas Beyer, James Bradbury, Naman Agrawal, Ben Poole, Igor Mordatch, Adam Roberts, and Jascha Sohl-Dickstein. Velo: Training versatile learned optimizers by scaling up, 2022.

[11] Abhinav Moudgil, Boris Knyazev, Guillaume Lajoie, and Eugene Belilovsky. Celo: Training versatile learned optimizers on a compute diet, 2025.

[12] Andrea Martin and Luca Furieri. Learning to optimize with convergence guarantees using nonlinear system theory, 2024.

[13] Camille Castera and Peter Ochs. From learning to optimize to learning optimization algorithms, 2025.

[14] Xiang Wang, Shuai Yuan, Chenwei Wu, and Rong Ge. Guarantees for tuning the step size using a learning-to-learn approach, 2021.

[15] Jialin Liu, Xiaohan Chen, Zhangyang Wang, Wotao Yin, and HanQin Cai. Towards constituting mathematical structures for learning to optimize, 2023.

[16] Qingyu Song, Wei Lin, and Hong Xu. Learning provably improves the convergence of gradient descent, 2025.

[17] Junjie Yang, Tianlong Chen, Mingkang Zhu, Fengxiang He, Dacheng Tao, Yingbin Liang, and Zhangyang Wang. Learning to generalize provably in learning to optimize, 2023.

[18] Wenqing Zheng, Tianlong Chen, Ting-Kuei Hu, and Zhangyang Wang. Symbolic learning to optimize: Towards interpretability and scalability, 2022.

[19] Niru Maheswaranathan, David Sussillo, Luke Metz, Ruoxi Sun, and Jascha Sohl-Dickstein. Reverse engineering learned optimizers reveals known and novel mechanisms, 2021.

[20] Zhenghan Fang, Sam Buchanan, and Jeremias Sulam. What's in a prior? learned proximal networks for inverse problems, 2024.

[21] Bingheng Li, Linxin Yang, Yupeng Chen, Senmiao Wang, Qian Chen, Haitao Mao, Yao Ma, Akang Wang, Tian Ding, Jiliang Tang, and Ruoyu Sun. Pdhg-unrolled learning-to-optimize method for large-scale linear programming, 2024.

[22] Alex Oshin, Rahul Vodeb Ghosh, Augustinos D Saravanos, and Evangelos Theodorou. Deep flexQP: Accelerated nonlinear programming via deep unfolding. In *The Fourteenth International Conference on Learning Representations*, 2026.

[23] Kaifeng Lv, Shunhua Jiang, and Jian Li. Learning gradient descent: Better generalization and longer horizons, 2017.

[24] Olga Wichrowska, Niru Maheswaranathan, Matthew W. Hoffman, Sergio Gomez Colmenarejo, Misha Denil, Nando de Freitas, and Jascha Sohl-Dickstein. Learned optimizers that scale and generalize, 2017.